Rec^2: Embedding Table Reconstruction for Deep Recommender Systems

Xianquan Wang*
University of Science and Technology of China
Hefei, China
wxqcn@mail.ustc.edu.cn

Zhaocheng Du*
Huawei Noah's Ark Lab
Shenzhen, China
zhaochengdu@huawei.com

Song-Li Wu
Tsinghua University
Shenzhen, China
wsl24@mails.tsinghua.edu.cn

Zirui Liu
University of Science and Technology of China
Hefei, China
liuzirui@mail.ustc.edu.cn

Haotian Zhang
University of Science and Technology of China
Hefei, China
sosweetzhang@mail.ustc.edu.cn

Jintao Zhang
University of Science and Technology of China
Hefei, China
zjttt@mail.ustc.edu.cn

Jieming Zhu
Huawei Noah's Ark Lab
Shenzhen, China
jamie.zhu@huawei.com

Shuai Wang
Yale University
New Haven, USA
shuai.wang.sw2572@yale.edu

Kai Zhang†
University of Science and Technology of China
Hefei, China
kkzhang08@ustc.edu.cn

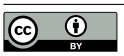
Abstract

Recommender systems are a pivotal component of modern Internet services. Among them, embedding tables play a crucial role in predicting user preferences and behaviors. However, existing studies mainly focus on improving downstream recommender models, while neglecting the structural importance of the embedding table itself. As a result, they fail to fully exploit the potential of feature fields and their interactions. To address this limitation, we propose the Rec^2 framework (**Rec**onstruction for **Rec**ommender Systems), which aims to optimize the organization of embedding tables by efficiently determining which features should be split or aggregated. We introduce a novel method that estimates feature importance based on second-order Taylor expansion contributions, enabling precise identification of features that benefit from splitting or pre-constructed interactions. Moreover, we further integrates second-order estimation with a global heap mechanism to efficiently manage interactions among key features and their aggregations. By improving the structure of the embedding table, Rec^2 significantly enhances recommender system performance and shows great potential for advancing personalized recommendation. The code is available at https://github.com/xqwustc/Rec2.

CCS Concepts

• **Information systems** → **Recommender systems**.

*Equal Contribution.
†Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

KDD 2026, Jeju Island, Republic of Korea.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817638

Keywords

Feature Construction, Data Mining, Recommender Systems

ACM Reference Format:
Xianquan Wang, Zhaocheng Du, Song-Li Wu, Zirui Liu, Haotian Zhang, Jintao Zhang, Jieming Zhu, Shuai Wang, and Kai Zhang. 2026. Rec^2: Embedding Table Reconstruction for Deep Recommender Systems. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026), August 9–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3770855.3817638

1 Introduction

Recommender systems [40] play a critical and intuitive role in e-commerce, advertising [24, 33, 34, 37] and ranking [23, 25, 26]. The core idea of them is to predict user clicks and preferences based on user data, item data, and contextual information [19, 42] (such as location and time). Deep learning-based recommenders typically treat these feature inputs as tokens [14, 16], map them to unique latent representations via embedding tables, and feed them into deep networks [3, 10]. While many pioneering studies focus on modifying the deep networks [38], they often overlook the embedding table [2], which serves as the foundation. Therefore, properly designing the embedding table is crucial for prediction.

The conventional approach involves constructing an embedding table based on the raw input data [6]. Raw features are divided into different feature fields, each containing multiple feature values. Each feature value is mapped to a corresponding latent representation through its feature field's embedding*. However, this seemingly appropriate method does not fully exploit the potential of the features. **Firstly**, not all feature field structures are necessarily the most suitable. For example, different features within the same field

*A *feature field* corresponds to **a specific column** in a table, while a *feature* (more precisely, a *feature value*) denotes **a particular value** within that field. For example, if the feature field is `occupation`, a possible feature value is `doctor`.

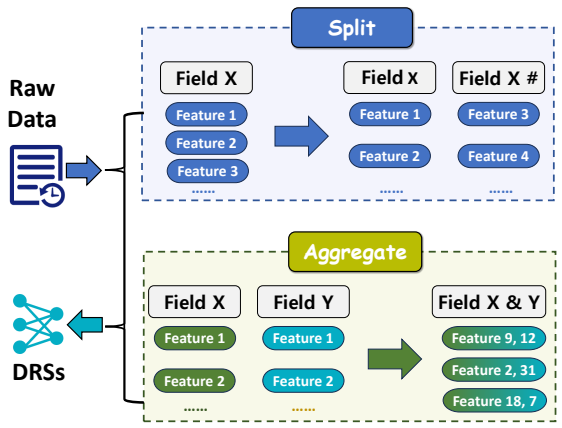


Figure 1: Split and Aggregate Operation. The upper part of the figure illustrates how Split decomposes values belonging to a single feature field, while the lower part demonstrates how Aggregate combines values to form new feature fields.

typically undergo similar linear transformations after embedding, which may not be optimal. Take a real-world case, the *mobile phone* feature field, isolating the features of expensive phones into a separate field may better model the *affluent* user group. **On the other hand**, the embedding table itself cannot provide prior knowledge of important interactions. Many existing models attempt to capture user behavior patterns more effectively through feature fields crossing [2, 15] or combinations [2, 33]. However, these automatic *feature cartesianizing methods* are overly idealized and may introduce significant noise (*a.k.a.* useless extra product). By constructing feature value-level interactions during the embedding stage, the model could more efficiently [12, 17, 18] learn these relationships, thus aiding the learning process.

Therefore, an ideal embedding table should meet requirements in both aspects. In terms of feature fields, each field in the embedding table should allow its features to achieve better results compared to when the feature subset is split into new feature fields. We term this the **Field Completeness** property. In terms of pre-constructed feature interactions, the table should provide important interactions at the embedding layer, enabling the model to directly model these new interaction features, which we refer to as the **Interaction Pre-construction** property. In order to achieve the above goals, we need to perform two operations on the embedding table: one is the **Split** operation, which separates features suitable for independent modeling from their original fields and assigns them to new fields for modeling; the other is the **Aggregate** operation, which aggregates important feature interactions into new feature fields to enhance the modeling capacity of the model. Nevertheless, achieving this goal is far from easy. Features that tend to be split out and belong to new feature fields contribute significantly to the prediction results. However, a reasonable measurement strategy is required to distinguish them through the model's performance and form new fields. For important interactions, efficiently measuring the pair-wise contributions of features is necessary to construct new feature fields for these features.

To address the above issues, we propose the **Rec**^{2} (Embedding Table **Rec**onstruction for **Rec**ommender Systems) framework, which aims to efficiently determine which features should undergo split operations and which should be aggregated. Specifically, we measure whether each feature (*i.e.*, feature value) should undergo one of these two operations or neither, by examining the second-order contribution of the Taylor expansion of the loss function. For features that need to be split, we introduce an additional identity column during pre-training to indicate the meaning of "split," functioning like a magnet. If a feature interacts highly with it, we consider that the feature is more likely to be split or that splitting it brings greater performance benefits from the expected perspective. For features that require pre-constructed interactions, we measure the pair-wise importance of the features. Both parts of this strategy are unified through the calculation of the second-order Taylor expansion contribution: one is the joint importance of a single feature and the identity feature, and the other is the joint importance of feature-values. To make the importance calculation more efficient, we use a global heap to accelerate the second-order gradient estimation and feature importance ranking process, which would make the framework more practical. Overall, our contributions are as follows:

- We introduce an efficient framework that quickly selects features to be split into separate columns and identifies key feature interactions, laying the foundation for embedding reconstruction.
- We present a method to speed up estimation by applying Taylor expansion to rank the importance of features to be split, and using global heap to efficiently aggregate important feature interactions.
- We conduct extensive experiments on two public datasets, showing the effectiveness of our method.

2 Related Work

2.1 Deep Recommender Systems (DRSs)

Delivering items that precisely match user preferences is the core mission of DRSs [22]. Historically, capturing complex user interests has heavily relied on designing sophisticated feature interaction modules. From the implicit MLP interactions in **DNN** [1] to the explicit crossing networks in **DCN** [31] and **DeepFM** [7], the evolution of DRSs is marked by continuous architectural refinements. Furthermore, diverse mathematical operators have been integrated into this process, ranging from the inner/outer products in IPNN/OPNN [29] to recent advancements like the fast-exponentiation-based FCN [14] and the *Khatri–Rao* product in QNN [16]. Despite their empirical success, these methods share a fundamental limitation: they are fundamentally constrained by specific, predefined assumptions about feature interaction patterns. Instead of engineering more complex interaction functions within the model, we explore a data-centric alternative. By explicitly splitting and aggregating features before they enter the model, we can independently introduce topological knowledge of feature organization, providing universally applicable feature property information without relying on restrictive interaction assumptions.

2.2 Feature Engineering in DRSs

Existing works predominantly targets single feature selection [11]. Various strategies have been proposed to evaluate field-level importance, ranging from gating (AutoField [36]) and permutation tests (PFI [5]) to sample-aware network weighting (AdaFS [21], MvFS [13]) and bias mitigation (FairFS [32]). Despite extensions into multi-task scenarios [35], these approaches are inherently confined to selecting existing features rather than systematically reconstructing the embedding table [9]. Although recent advances like LightCS [2] and TayFCS [33] have explored higher-order feature combinations, they primarily focus on search and selection strategies. Rec^2 fundamentally reconstructs the embedding space from a gradient perspective, practically splitting critical individual features while aggregating strongly interacting value pairs.

3 Preliminaries and Problem Setup

3.1 DRS Pipeline

Formally, a recommendation dataset encompassing n distinct feature fields is defined as $\mathcal{D} = (\mathbf{X}, \mathbf{y})$. Within this dataset, the multi-field input vector is denoted by $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_i, \ldots, \mathbf{x}_n]$, where each $\mathbf{x}_i$ acts as the sparse, one-hot encoded representation of the i-th field. The corresponding ground-truth targets for all instances in the dataset of size $|\mathcal{D}|$ are structured as a binary vector $\mathbf{y} \in \{0,1\}^{|\mathcal{D}|}$.

To map the high-dimensional sparse inputs into a dense continuous space, standard Deep Recommender Systems (DRSs) utilizing an embedding paradigm [27, 30]. Specifically, a sparse feature $\mathbf{x}_i$ is projected into a continuous vector $\mathbf{e}_i$ via the lookup operation $\mathbf{e}_i = \mathbf{V}_i \mathbf{x}_i$, relying on the field-specific embedding dictionary $\mathbf{V}_i$. These individual dense vectors are subsequently concatenated to construct a comprehensive dense representation $\mathbf{E} = [\mathbf{e}_1, \ldots, \mathbf{e}_i, \ldots, \mathbf{e}_n]$.

Upon obtaining the integrated representation $\mathbf{E}$, it is forwarded into downstream models [8, 41]. The concatenated dense features are subjected to complex operations, including multi-order crossover mechanisms, linear combinations [28], and non-linear activations [20], to capture latent interaction patterns. By abstracting this forward propagation as a network mapping function g parameterized by Θ, the final predicted probability $\hat{\mathbf{y}}$ can be formulated as $\hat{\mathbf{y}} = g(\mathbf{E}; \Theta)$.

The optimization of both the embedding spaces $\mathbf{E}$ and the network weights Θ is driven by minimizing the empirical risk. In click-through rate prediction tasks, the standard choice is the Binary Cross-Entropy (BCE) objective. Over the entire dataset, the discrepancy between the algorithmic predictions $\hat{y}_i$ and the true binary labels y_i is evaluated as:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}; \mathbf{E}, \Theta) = -\frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]. \quad (1)$$

3.2 Embedding Table Reconstruction

3.2.1 Split. The split operation involves partitioning a feature into multiple subcomponents, each of which is handled independently. This allows for more fine-grained modeling of the feature or feature interaction, capturing additional patterns that may be overlooked when treated as a single entity. In DRSs, these splits are treated as separate features, each encoded independently and fed into the model's input layer.

Formally, for a set of original feature fields $\mathcal{F} = \{\mathcal{F}_1, \mathcal{F}_2, \ldots, \mathcal{F}_n\}$, where each $\mathcal{F}_i$ represents a feature field consisting of a set of features, the split operation generates a new set of feature fields $\mathcal{F}' = \{\mathcal{F}'_1, \mathcal{F}'_2, \ldots, \mathcal{F}'_m\}$, where each $\mathcal{F}'_i$ is a subset of features from the original set $\mathcal{F}$. These new feature fields are disjoint and their union equals the original feature fields:

$$\mathcal{F}'_i \cap \mathcal{F}'_j = \emptyset, \quad \forall i \neq j, \quad (2)$$

and

$$\bigcup_{i=1}^{m} \mathcal{F}'_i = \bigcup_{i=1}^{n} \mathcal{F}_i, \quad (3)$$

where each subset represents a partition of a feature or a combination of features, and these subsets are processed independently.

3.2.2 Aggregate. The aggregate operation involves making feature Cartesian product from the original feature fields $\mathcal{F} = \{\mathcal{F}_1, \mathcal{F}_2, \ldots, \mathcal{F}_n\}$ to generate new feature fields $\mathcal{F}' = \{\mathcal{F}'_1, \mathcal{F}'_2, \ldots, \mathcal{F}'_m\}$. Specifically, for each pair of feature fields $\mathcal{F}_i$ and $\mathcal{F}_j$, the operation generates potential new features representing interactions between features in $\mathcal{F}_i$ and $\mathcal{F}_j$. However, instead of including all possible feature interactions, only a subset of the most informative or important interactions is selected to form the new feature fields.

Formally, for each pair $\mathcal{F}_i$ and $\mathcal{F}_j$, the aggregate operation generates a set of feature interactions $C(\mathcal{F}_i, \mathcal{F}_j)$, which is the Cartesian product of the features in $\mathcal{F}_i$ and $\mathcal{F}_j$. However, instead of using all the possible feature interactions, we select a subset of these interactions, denoted by $\mathcal{F}'_k$, to form the new feature fields:

$$\mathcal{F}'_k \subseteq \mathcal{F}_i \times \mathcal{F}_j, \quad \forall 1 \leq i < j \leq n, \quad (4)$$

where $\mathcal{F}'_k$ is the subset of the Cartesian product $\mathcal{F}_i \times \mathcal{F}_j$ representing the most important or informative feature interactions selected to form the new feature fields.

4 Framework

The whole pipeline is shown in Figure 2. Our framework primarily measures importance through Taylor expansion. Specifically, a recommender model is trained based on the training set. Then, as shown in Equation 1, we rewrite it as a function of feature embeddings, treating the prediction, labels, and parameters of the well-trained network model as constants. Then the loss function will be:

$$\mathcal{L}(\mathbf{E}) \approx \mathcal{L}(\tilde{\mathbf{E}}) + \sum_{e_v \in \cup \mathcal{F}_i} \frac{\partial \mathcal{L}}{\partial e_v} \delta e_v + \frac{1}{2} \sum_{i \neq j}^{|\mathcal{F}|} \sum_{e_{v_i} \in \mathcal{F}_i} \sum_{e_{v_j} \in \mathcal{F}_j} \frac{\partial^2 \mathcal{L}}{\partial e_{v_i} \partial e_{v_j}} \delta e_{v_i} \delta e_{v_j}, \quad (5)$$

where $\mathcal{L}(\tilde{\mathbf{E}})$ is the value of the loss function evaluated at the reference point $\tilde{\mathbf{E}}$, around which the expansion is conducted. The term $\frac{\partial \mathcal{L}}{\partial e_v}$ denotes the first derivative of the loss with respect to the embedding e_v, where e_v corresponds to the embedding of the feature value v from features in all fields. This derivative measures how sensitive the loss function is to changes in e_v. The perturbation δe_v represents a small change in e_v around the reference point $\tilde{e}_v$, with $\delta e_{v_i} = e_{v_i} - \tilde{e}_{v_i}$. The second derivative $\frac{\partial^2 \mathcal{L}}{\partial e_{v_i} \partial e_{v_j}}$ measures the interaction between the embeddings e_{v_i} and e_{v_j}, capturing how the joint effect changes in these two embeddings affect the loss. The product $\delta e_{v_i} \delta e_{v_j}$ represents the combined effect of small changes

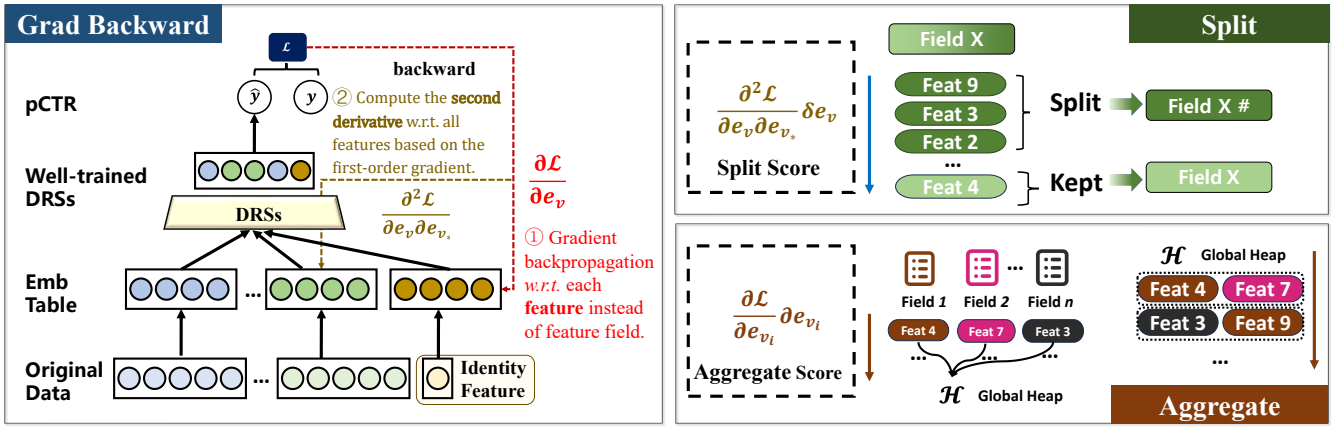


Figure 2: The left side of the figure shows the gradient backpropagation process. The red part indicates gradients from the loss function backpropagated to each feature, including identity features. The orange part shows the second-order derivatives of the loss with respect to the identity feature gradients and all features. The right side shows how to use and calculate two kinds of scores.

in both embeddings on the loss function. Together, these terms provide a second-order approximation of the loss function around $\tilde{\mathbf{E}}$. Therefore, for features, their contribution primarily comes from the first and second-order terms of the Taylor expansion. The first-order term corresponds to the individual importance of each feature, which is less of a focus. The Split and Aggregate operations mainly concentrate on analyzing the **second-order** Taylor term. The goal of our method is to get the ranking of Split score and Aggregate Score, and we could use K_1 and K_2, respectively, to control the number of feature split and aggregation.

4.1 Split Stage

To distinguish features that need to be split, we introduce an additional feature field $\mathcal{F}_*$ during the pre-training phase, added to the original feature field set $\mathcal{F} = \{\mathcal{F}_1, \mathcal{F}_2, \ldots, \mathcal{F}_n\} \cup \mathcal{F}_*$. $\mathcal{F}_*$ represents an identity column containing a single feature value, indicating "is_need_split". If a feature value shows a strong interaction signal, we consider this feature more likely to be split into a separate column and play a role in a new feature field. As shown in Equation 5, the Split scores for all feature values are:

$$\sum_{e_v \in \cup \mathcal{F}_i} \frac{\partial^2 \mathcal{L}}{\partial e_v \partial e_{v_*}} \delta e_v \delta e_{v_*}, \tag{6}$$

where e_{v_*} represents the embedding of a feature from the identity feature field, which contains a special feature value. Here, $\delta e_{v_*} = e_{v_*} - \tilde{e}_{v_*}$ and $\tilde{e}_{v_*}$ is the reference point. For $\delta e_v = e_v - \tilde{e}_v$, We choose the mean embedding of the feature field as the reference, and since $\mathcal{F}_*$ contains only one feature value, we select the zero embedding as its reference point. From this, we can observe that δe_{v_*} is the same for all feature values. Therefore, this part does not affect the process of ranking the Split scores. As a result, the Split score for each feature value v can be expressed as:

$$\frac{\partial^2 \mathcal{L}}{\partial e_v \partial e_{v_*}} \delta e_v, \tag{7}$$

Algorithm 1 Split Score Calculation

Input:
Original set of feature fields $\mathcal{F} = \{\mathcal{F}_1, \mathcal{F}_2, \ldots, \mathcal{F}_n\}$
A pre-trained model with loss function $\mathcal{L}$ (for the validation dataset) and embedding vector e_v for each feature value v.

Initialization:
$\tilde{e}_{v_*} \leftarrow \mathbf{0}$; $Scores \leftarrow []$.

Global Computation:
Calculate the global first-order derivative w.r.t e_{v_*}: $\frac{\partial \mathcal{L}}{\partial e_{v_*}}$

for all feature field $\mathcal{F}_i \in \mathcal{F}$ **do**
  Calculate the mean embedding for the field: $\bar{e}_{\mathcal{F}_i} \leftarrow \frac{1}{|\mathcal{F}_i|} \sum_{v' \in \mathcal{F}_i} e_{v'}$
  for all feature value $v \in \mathcal{F}_i$ **do**
    Define the reference point for v: $\tilde{e}_v \leftarrow \bar{e}_{\mathcal{F}_i}$
    Calculate the embedding difference: $\delta e_v \leftarrow e_v - \tilde{e}_v$
    Calculate the second-order derivative based on the global first-order derivative: $\frac{\partial}{\partial e_v}\left(\frac{\partial \mathcal{L}}{\partial e_{v_*}}\right)$
    Calculate the Split Score: $S_v \leftarrow \frac{\partial^2 \mathcal{L}}{\partial e_v \partial e_{v_*}} \delta e_v$
    Append (v, S_v) to the $Scores$ list
  end for
end for
Sort the $Scores$ in descending order based on S_v

Output:
List $Scores$

where a higher score indicates that the corresponding feature v is more suitable to be split into a separate column, as in Figure 2. The pseudocode of it is also shown in Algorithm 1.

4.2 Aggregate Stage

Aggregate operations introduce additional useful feature interactions, whose importance can be characterized by the second-order interaction importance between features. Specifically, for the embedding $\mathbf{e}_{v_i}$ of a feature value from field $\mathcal{F}_i$ and the embedding $\mathbf{e}_{v_j}$ from field $\mathcal{F}_j$, their interaction importance can be defined as:

$$\frac{\partial^2 \mathcal{L}}{\partial e_{v_i} \partial e_{v_j}} \delta e_{v_i} \delta e_{v_j}, \tag{8}$$

However, computing the second-order gradient of $\frac{\partial^2 \mathcal{L}}{\partial e_{v_i} \partial e_{v_j}}$ directly is intractable, as it requires fixing one feature value, computing its derivative, and then applying it across all values in other fields. In recommender systems, the number of features often reaches millions or tens of millions, making such operations computationally infeasible. Therefore, we need to approximate and accelerate the computation process. Fortunately, TayFCS [33] has already introduced the Information Matrix Equality (IME), which provides a statistically sound approximation of second-order gradients. Specifically, for any twice-differentiable (C^2) function such as the loss $\mathcal{L}$, the following holds:

$$\frac{\partial^2 \mathcal{L}}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}^T} \approx \mathbb{E}\left[\frac{\partial^2 \mathcal{L}}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}^T}\right] = -\mathcal{I}(\boldsymbol{\theta}). \tag{9}$$

The Fisher information $\mathcal{I}(\boldsymbol{\theta})$ is typically estimated using its first definition based on first-order derivatives, since gradients are generally easier and more stable to compute than second-order derivatives. The Fisher information has the definition:

$$\mathcal{I}(\boldsymbol{\theta}) = \mathbb{E}\left[\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}\right)\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}\right)^T\right], \tag{10}$$

in this way, when the dataset has many samples, the derivative w.r.t. both variables can be estimated by the product of their individual gradients. Backing to Equation 8, it can be estimated as:

$$\frac{\partial \mathcal{L}}{\partial e_{v_i}} \frac{\partial \mathcal{L}}{\partial e_{v_j}} \delta e_{v_i} \delta e_{v_j} = \frac{\partial \mathcal{L}}{\partial e_{v_i}} \delta e_{v_i} \frac{\partial \mathcal{L}}{\partial e_{v_j}} \delta e_{v_j}. \tag{11}$$

However, this is still not efficient enough for identifying most useful aggregated features. Computing gradient products for all pairs **among millions of feature values** leads to unacceptable time complexity. Therefore We introduce an iterative selection algorithm that leverages a priority queue to dynamically discover the top-K interactions without ever materializing the entire set of pairs $S_i S_j$, where:

$$S_i = \frac{\partial \mathcal{L}}{\partial e_{v_i}} \delta e_{v_i}. \tag{12}$$

4.2.1 Core Idea and Problem Transformation. The key insight is to transform the problem from an exhaustive search to a prioritized, iterative search. We can conceptualize all possible interaction scores, $S_i \cdot S_j$, as forming an $V \times V$ matrix. If we first sort the individual feature scores S_i, this conceptual matrix will exhibit a sorted structure—its rows and columns will be sorted. In such a matrix, the maximum value is located at a corner, and any subsequent high-ranking values are guaranteed to be adjacent to already discovered high-ranking values.

This structure allows us to perform a best-first search starting from the globally optimal solution. A max-heap (priority queue) is the ideal data structure for managing this search, as it always provides the most promising candidate for expansion. The algorithm consists of two main stages: a one-time preprocessing step and an iterative search phase.

First, is to establish the ordered structure required by our search. We compute the individual importance score S_i for all n features. Second, since importance is related to the magnitude of the score, we sort these scores based on their absolute values in **descending order**. Let the resulting array of scores be denoted by $\mathcal{S}_{\text{sorted}} = (S_1, S_2, \ldots, S_V)$, where the index j refers to the rank in the sorted list, such that $|S_1| \geq |S_2| \geq \cdots \geq |S_V|$. With the sorted scores, we can now efficiently find the top-K products. The search is conducted iteratively, managed by a max-heap $\mathcal{H}$ that stores candidate interactions. Each element in the heap is a tuple $(S_i S_j, i, j)$, where i, j are the 1-based indices in $\mathcal{S}_{\text{sorted}}$. A set $\mathcal{V}$ is used to track visited index pairs to prevent redundant computations.

The process begins by initializing the heap with the single, globally maximum interaction score, which is guaranteed to be $S_1 S_1 = |S_1 \cdot S_1|$. Thus, the initial state of the heap is $\mathcal{H}_0 = \{(S_1 S_1, 1, 1)\}$. The algorithm then proceeds for K iterations. In each iteration, the top candidate is extracted from the heap, and its neighboring entries in the conceptual score matrix—each of the form $S_i S_j$—are generated and inserted into the heap. This strategy ensures that the search systematically expands from the most promising regions of the solution space. However, one issue with this approach is that importance scores are calculated for features within the same feature field. This leads to a significant problem: when searching for another feature to pair with it as a promising aggregation, features within the same field must be excluded. This introduces an **additional neighbor search process**, where we continue to look for high-score features that do not belong to the same feature field. As a result, logarithmic time complexity can no longer be guaranteed. Therefore, we propose an improvement to this approach.

4.2.2 Avoiding Aggregation with Features in the Same Field. First, we preprocess all feature importance scores. We group all feature importance scores according to the feature field they belong to. Each field forms a list containing all of its feature scores. Based on this, we sort the score list within each feature field in **descending order**. After this step, we obtain n sorted score lists, denoted as $F_1, F_2, \ldots, F_n$. The goal is to find the top-K largest products from all possible cross-field aggregations $F_k[i] * F_l[j]$, where $k \neq l$.

To find the global top-K, we use a single Global Max-Heap $\mathcal{H}$ that aggregates the most promising feature interactions from all pairs of fields. Each element in the heap is a **5-tuple**: $(\text{score}, k, l, i, j)$, which represents the interaction score $S_{F_k[i]} \cdot S_{F_l[j]}$, the indices of the two feature fields (k and l), and the indices of the scores within those fields' sorted lists (i and j).

$\mathcal{H}$ is initialized by considering every pair of distinct feature fields. For each pair (k, l) where $k < l$, we compute the interaction score of their top-ranking features and push the corresponding tuple onto the heap:

$$\text{score} = S_{F_k[1]} \cdot S_{F_l[1]}, \tag{13}$$

the tuple $(\text{score}, k, l, 1, 1)$ is then pushed into the Global Max-Heap. After this initialization process, the heap contains $\frac{n(n-1)}{2}$ elements,

with each element representing the most promising feature aggregation from a unique pair of fields. This sets the stage for the global search.

Based on this, We further maintain a visited set to avoid redundant processing, which records the visited index pairs (i, j) for each field pair (k, l). We then perform K iterations, or until the heap is empty. Each time, the top element will be popped, as $(\text{score}, k, l, i, j)$. This is one of our global top-K results, and it will be added into the final score list. To continue exploring aggregation from this specific pair, we find its "neighbors" and add them to the Global Max-Heap to compete with candidates from other field pairs. The two kinds of neighbors are **Neighbor A (Next in F_k)**:

$$(F_k[i+1] * F_l[j], k, l, i+1, j), \tag{14}$$

and **Neighbor B (Next in F_l)**:

$$(F_k[i] * F_l[j+1], k, l, i, j+1). \tag{15}$$

For the candidate neighbor, if it has not been visited, then push its 5-tuple above into the $\mathcal{H}$ and mark the neighbor as visited. Then the iteration will continue. Repeating this process K times efficiently yields the global top-K aggregation scores between features from different fields. The pseudocode of it is shown in Algorithm 2.

4.3 Feature Fields Control

To better concentrate the features within specific feature fields rather than dispersing it across many, we introduce a hyperparameter $\tilde{m}$. This parameter represents the total number of new feature fields created through both splitting and aggregation. In other words, it defines the total number of added feature fields. We then greedily select the most important split and aggregation feature fields one by one. If including a candidate causes the total number of added feature fields to exceed $\tilde{m}$, that candidate is skipped; otherwise, it is added to the selected set. This process continues until the required number of split and aggregated features is obtained. This approach helps the model focus on truly useful feature fields based on actual needs, and avoids regenerating new features from already aggregated features within the same pair fields, which may waste the model's capacity.

4.4 Time Complexity Analysis

First, during the pretraining phase, the computational cost is mainly due to training with an embedding table that includes identity features. This cost is comparable to standard training. Next, the time complexity of gradient analysis on the validation set is considered. To compute the second-order gradient w.r.t. each feature value for the identity feature, we first compute the gradient with respect to the identity feature using the loss, and then use this gradient to compute the gradient for all features. The cost of the first gradient computation is negligible. The score S for each feature value can be obtained through a single gradient pass. Therefore, the time complexity of gradient analysis is essentially the complexity of two gradient backpropagations on the validation set.

Regarding the time complexity of Split and Aggregation, the importance of Split is determined through global sorting, which is $O(V \log V)$, where V is the total number of feature values. For Aggregation, the overall time complexity of the algorithm is $O(V \log V + n^2 + K \log(n^2 + K))$, where n is the number of feature fields, and K is the number of top interactions to find. This complexity consists of three parts: an initial cost of $O(V \log V)$ for grouping and sorting all V feature values; a cost of $O(n^2)$ to initialize the max-heap with a candidate from each of the $\frac{n(n-1)}{2}$ field pairs; and a cost of $O(K \log(n^2 + K))$ for the K target aggregated features.

Algorithm 2 Searching Top Important Aggregated Features

Input:
  Feature importance scores grouped by field: $F_1, F_2, \ldots, F_n$
  Number of interactions to find: K_2

Preprocessing:
for all field list F_i from $i = 1$ to n **do**
  Sort F_i in descending order.
end for

Initialization:
Initialize a global max-heap $\mathcal{H}$
Initialize a result list $Results \leftarrow []$
Initialize a visited set $Visited \leftarrow \emptyset$
for $k \leftarrow 1$ to $n-1$ **do**
  for $l \leftarrow k+1$ to n **do**
    if F_k and F_l are not empty **then**
      $score \leftarrow F_k[1] \cdot F_l[1]$
      $\mathcal{H}$.push((score, k, l, 1, 1))
      $Visited$.add((k, l, 1, 1))
    end if
  end for
end for

while $|Results| < K_2$ and $\mathcal{H}$ is not empty **do**
  $(score, k, l, i, j) \leftarrow \mathcal{H}$.pop()
  $Results$.add((score, k, l, i, j))
  ▷ Neighbor A: advance in field k
  if $i+1 \le |F_k|$ and $(k, l, i+1, j) \notin Visited$ **then**
    $new_score \leftarrow F_k[i+1] \cdot F_l[j]$
    $\mathcal{H}$.push((new_score, k, l, $i+1$, j))
    $Visited$.add((k, l, i+1, j))
  end if
  ▷ Neighbor B: advance in field l
  if $j+1 \le |F_l|$ and $(k, l, i, j+1) \notin Visited$ **then**
    $new_score \leftarrow F_k[i] \cdot F_l[j+1]$
    $\mathcal{H}$.push((new_score, k, l, i, $j+1$))
    $Visited$.add((k, l, i, j+1))
  end if
end while

Return: $Results$

5 Experiment

In this section, we will detail the performance of Rec^2. We use different feature engineering methods from various perspectives to conduct this evaluation, and the experiments will answer the following questions:

- **RQ1:** Can Rec^2 perform better than existing methods?

Table 1: Statistics of two public datasets.

Dataset	#Fields	#Training	#Validation	#Test	Positive%
Frappe	10	202,027	57,722	28,860	33.46%
iPinYou	16	13,195,935	2,199,323	4,100,716	0.08%

- **RQ2:** Does the reconstructed embedding table built on Rec^2 help the training process?
- **RQ3:** How do hyperparameters affect Rec^2?
- **RQ4:** Can the embedding table reconstruction be transferred to other models (*e.g.*, DCN to DNN)?
- **RQ5:** How does each component contribute to the final performance of Rec^2?

5.1 Experiment Setup

5.1.1 Base Models. Our experiments are based on three classic models to understand the effectiveness of our method across different models. We use **DNN**, **DCN** and **Wide & Deep** as base models. These models include deep-only models and those that combine deep models with novel Cross networks. This helps to evaluate the gains of our method across different feature crossing models.

5.1.2 Dataset Details. We use the most commonly adopted datasets in DRSs: **Frappe** and **iPinYou**. We evaluate the performance of various feature engineering and selection methods on baseline models with these datasets. Following the FuxiCTR repo[†], we split the Frappe dataset into training, validation, and test sets with a ratio of 7:2:1, and for iPinYou, the detailed statistics are shown in Table 1. Here is a detail description of the two public datasets we used for evaluation.

- **Frappe[‡]:** This dataset includes context-aware application usage records, characterized by 10 distinct categorical feature fields. They are all non-anonymous feature fields like *user*, *item*, *cost*, *weather*, *country*, and so on.
- **iPinYou[§]:** It includes all training datasets and leaderboard testing datasets for each season, covering DSP bidding, impression, click, and conversion logs. The reserved testing datasets used for final evaluation are withheld by iPinYou.

Because our method focuses on feature value-level splitting and aggregation, these two datasets contain only categorical features and are therefore well suited for validating our idea.

5.1.3 Comparative Methods. The methods we compare are: **Random**, **AutoField**, **AdaFS**, **MvFS** and **TayFCS**, which are detailed in the related work section. Here, "Random" does not refer to fully random selection. Instead, a large value is pre-set for K (i.e., K_1 and K_2), and scores are first computed using Rec^2. Then, the resulting score list is shuffled (rather than sorted in descending order), and important features are selected from it. Although this introduces randomness, the selection is still guided by Rec^2-based importance. For AutoField, we score a batch feature pairs at a time and then aggregate and rank the results. For TayFCS, to ensure fairness, we use TayScorer to obtain the combination score without addtional LRE. For AdaFS and MvFS, they assign instance-level weights to each feature field.

[†]Thanks to the public repo: https://github.com/reczoo/FuxiCTR
[‡]https://www.baltrunas.info/context-aware
[§]http://contest.ipinyou.com/

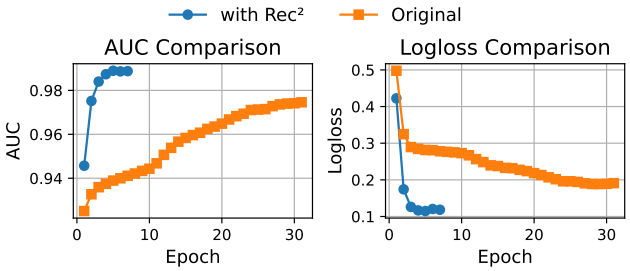


Figure 3: Metrics comparison on the validation dataset. With Rec^2, the base model converges faster.

5.1.4 Metrics and General Hyperparameters. Following previous works [39], we use AUC [4] and Logloss to evaluate model performance. About the Relative Improvement (RelImp), it is defined as $(\frac{\text{AUC(Method AUC)}-0.5}{\text{AUC(Base Model AUC)}-0.5} - 1) \times 100\%$, which quantifies the relative performance improvement compared to the original base model. The training is set to a maximum of $T_{\max} = 100$ epochs with early stopping if AUC-Logloss does not improve for two consecutive epochs. Note that an improvement in AUC or a decrease in Logloss of **more than 0.001** is considered significant [7, 29].

For hyperparameters, we use a learning rate of 1e-3 and a batch size of 10,000 to pre-train the three models on Rec^2. For the Frappe and iPinYou datasets, the number of hidden neurons used in the pre-trained models are [500, 500, 500] or [400, 400, 400]. For K_1, we search the best one within $0.01V$ and $0.001V$. The embedding size is set as 16 for each dataset. These hyperparameters can be used directly in the open-source framework FuxiCTR [43, 44].

5.2 Overall Performance (RQ1)

The main results are shown in Table 2. Overall, our proposed Rec^2 demonstrates strong predictive performance. Under the same added feature fields, Rec^2 achieves the best AUC and Logloss results across both datasets. Compared with other methods, Rec^2 clearly improves prediction accuracy. Even the Random variant, which shuffles Rec^2 outputs, often enhances the base model's performance, indicating that Rec^2 effectively identifies features that should be split and aggregated. Although AutoField and TayFCS can select important feature crossings, its overall performance is inferior to ours. This may be because it only performs crossings between feature fields, and such coarse granularity introduces redundancy, limiting improvement compared to Rec^2. Meanwhile, AdaFS and MvFS do not introduce new feature fields but instead reweight existing ones at the instance level. Their performance gain is smaller than that from adding new feature fields, suggesting that learning weights alone is less effective than reconstructing embeddings in the table beforehand, as the latter helps capture interaction patterns.

In addition, the performance gain brought by Rec^2 varies across different datasets and base models. For example, on the Frappe dataset, the improvement is most pronounced with the DCN model, while it is less significant with Wide & Deep compared to the other two base models. This may be because DCN already possesses some feature-crossing capability, though weaker than that of Wide &

Table 2: Performance comparison of different feature engineering methods. The symbols * and ** denote the t-test significance levels with $p \leq 0.05$ and $p \leq 0.01$ of Rec^2 compared to the best performing baseline. In this experiment with Random, AutoField and Rec^2, we add 5 more feature fields (split and aggregate) for Frappe dataset. As for iPinYou, we add 10 more feature fields.

Model	Method	DNN			DCN			Wide & Deep		
		AUC↑	Logloss↓	RelImp↑	AUC↑	Logloss↓	RelImp↑	AUC↑	Logloss↓	RelImp↑
Frappe	Original	0.9770	0.1793	–	0.9732	0.1889	–	0.9796	0.1710	–
	Random	0.9823	0.1373	1.11%	0.9836	0.1351	2.20%	0.9826	0.1370	0.63%
	AutoField(2022)	0.9826	0.1206	1.17%	0.9428	0.2757	-6.42%	0.9834	0.1198	0.79%
	AdaFS (2022)	0.9702	0.2032	-1.43%	0.9772	0.1811	0.85%	0.9706	0.2064	-1.88%
	MvFS (2023)	0.9780	0.1788	0.21%	0.9777	0.1778	0.95%	0.9757	0.1854	-0.81%
	TayFCS (2025)	0.9868	0.1263	2.05%	0.9871	0.1246	2.94%	0.9867	0.1282	1.48%
	Rec^2 (Ours)	**0.9880****	**0.1202****	**2.31%**	**0.9878****	**0.1196****	**3.09%**	**0.9877****	**0.1243****	**1.69%**
iPinYou	Original	0.7783	0.5561	–	0.7805	0.5550	–	**0.7810**	**0.5551**	–
	Random	0.7540	0.5736	-8.73%	0.7517	0.5649	-10.27%	0.7498	0.5681	-11.10%
	AutoField(2022)	0.7786	0.5576	0.11%	0.7793	0.5566	-0.43%	0.7795	0.5557	-0.53%
	AdaFS (2022)	0.7794	0.5567	0.40%	0.7809	0.5557	0.14%	0.7800	0.5558	-0.36%
	MvFS (2023)	0.7795	0.5573	0.43%	0.7814	0.5551	0.32%	0.7800	0.5558	-0.36%
	TayFCS (2025)	0.7793	0.5575	0.36%	0.7811	0.5557	0.21%	0.7777	0.5556	-1.17%
	Rec^2 (Ours)	**0.7803****	**0.5549****	**0.72%**	**0.7818***	**0.5546***	**0.46%**	0.7752	0.5583	-2.06%

Deep. When combined with Rec^2, DCN's crossing ability can be better exploited through the reconstructed embedding table, leading to the greatest relative improvement. In contrast, the overall improvement on the iPinYou dataset is smaller. This may be because the embedding table of iPinYou is already well-suited for deep models to learn effectively, and the dataset contains a larger number of original feature fields, allowing the base models' inherent modeling ability to perform adequately.

Regarding RelImp, Rec^2 consistently yields significant improvements across all datasets. For example, on the Frappe dataset, the average relative improvement reaches 2.36%, indicating that the original embedding table in Frappe suffers from noticeable structural flaws. On the iPinYou dataset, the gain is smaller, and Rec^2 does not perform as well, which may be due to the base model architecture being unsuitable. Interestingly, better original performance does not necessarily lead to greater gains when using the reconstructed embedding tables from Rec^2. For instance, on Frappe, DNN's original AUC is lower than that of Wide & Deep, yet it achieves the highest performance after applying Rec^2. These results further confirm that the performance upper bound of a model is highly dependent on the quality of the embedding table design.

5.3 Convergence Speed (RQ2)

In this subsection, we focus on the impact of table reconstruction on training efficiency. As shown in Figure 3, we conduct experiments on the Frappe dataset using the DCN model, comparing the convergence speed, validation AUC, and Logloss between the model with Rec^2 and the original one.

Two main observations can be made from the results. First, applying Rec^2 clearly accelerates model convergence—the required number of epochs decreases from 30+ to 7—indicating that the reconstructed table facilitates more efficient learning. Second, for any given epoch, the model with Rec^2 consistently achieves higher AUC and lower Logloss. Even in the first epoch, Rec2 achieves a significantly lower Logloss (from 0.5 to 0.42), indicating that good embedding engineering can provide the model with a favorable starting point for training. Therefore, we could infer that the reconstructed table helps the model build stronger associations between prediction outcomes and user–item information, leading to overall better predictive performance.

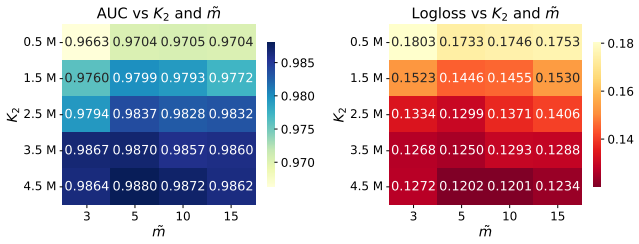


Figure 4: Hyperparameter analysis w.r.t. K_2 and $\tilde{m}$.

5.4 Hyperparameter Analysis (RQ3)

Rec^2 mainly has two hyperparameters: one, denoted as $\tilde{m}$, which controls the total number of feature fields; and another, denoted as K_2, which controls the number of aggregated features to be selected. The results in Figure 4 reveal two clear patterns in how K_2 and $\tilde{m}$ affect model performance. From the figure, we could get that increasing K_2 consistently improves model performance in both AUC and Logloss. The improvement is most noticeable when K_2 increases from a small to a moderate value, after which performance becomes stable. This indicates that a larger number of aggregated features enhances feature expressiveness up to a threshold, beyond which additional features bring little benefit. This also suggests that the model often reaches its best performance when K_2 is close to 4.5 million, possibly because the neglected aggregated features are somewhat redundant.

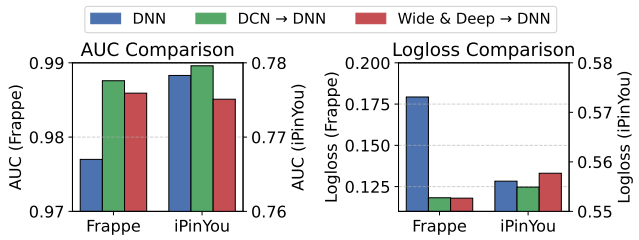


Figure 5: Transferring study for Frappe and iPinYou.

In addition, the influence of $\tilde{m}$ shows a non-monotonic trend. Increasing $\tilde{m}$ initially improves performance, as the added aggregated features provide more learning information, making the model easier to train. However, a large $\tilde{m}$ can cause the feature fields to become too dispersed, negatively impacting performance. This pattern suggests that a moderate level of feature selection complexity offers the best balance between interaction richness and the risk of overfitting.

5.5 Transfer Study (RQ4)

In this section, we investigate whether the important features identified by different base models can be transferred across models. Specifically, we examine whether the split and aggregation schemes derived from one base model can be applied to another to improve its performance. Here, we focus on testing whether the schemes obtained from DCN and Wide & Deep can enhance the performance of the most naive DNN model. Experiments are conducted on the Frappe and iPinYou datasets, with AUC and Logloss on the test set used as evaluation metrics. As shown in Figure 5, applying the embedding table reconstruction method—using DCN and Wide & Deep as base models—to DNN improves prediction performance on Frappe. However, on iPinYou, only the transfer from DCN results in a beneficial gain. The blue bars represent the original DNN model results, as shown in Table 2.

On Frappe, the AUC of the DNN model increases from 0.9770 to 0.9876 (from DCN) and 0.9859 (from Wide & Deep), with a noticeable decrease in Logloss. A similar trend is observed on iPinYou. On iPinYou, since Wide & Deep itself did not provide additional gains, the transfer did not yield better results. However, the embedding table structure transferred from DCN benefited DNN training, indicating that the learning effects between different models share significant commonality.

5.6 Ablation Study (RQ5)

In this section, we analyze how each component of Rec^2 affects the final prediction performance. Specifically, we evaluate three variants: (1) ***w/o Split***: no feature is split into new feature fields; (2) ***w/o Agg***: no aggregated features are used; (3) ***w/o*** $\tilde{m}$: the number of new feature fields is not controlled by $\tilde{m}$. That is, once the number of aggregated features is set, all resulting new fields are retained regardless of how many are generated.

As shown in Table 3, the full Rec^2 model achieves the best performance on both datasets with DNN and DCN. Among all components, aggregated features contribute the most. For example, on Frappe with DNN, removing aggregated features leads to a 4% drop in performance, even worse than the original model, indicating

Table 3: Ablation study for Rec^2.

	Model	Metric	Method			
			w/o Split	***w/o Agg***	***w/o*** $\tilde{m}$	**Rec^2**
Frappe	DNN	AUC ↑	0.9875	0.9667	0.9868	**0.9880**
		Logloss ↓	**0.1181**	0.2143	0.1238	0.1202
	DCN	AUC ↑	0.9868	0.9739	0.9861	**0.9878**
		Logloss ↓	**0.1173**	0.1866	0.1231	0.1196
iPinYou	DNN	AUC ↑	0.7787	0.7676	0.7547	**0.7803**
		Logloss ↓	0.5552	0.5612	0.5643	**0.5549**
	DCN	AUC ↑	0.7786	0.7627	0.7603	**0.7818**
		Logloss ↓	0.5561	0.5641	0.5645	**0.5546**

that using split operations alone without aggregation may harm the model. Removing the split operation also causes performance degradation, though to a lesser extent than removing aggregation. Additionally, excluding the control of feature field number ($\tilde{m}$) results in noticeable performance loss. In our experiments, we found that without this control, aggregated features are spread across more than 20 fields, making it difficult for the model to learn effective feature interactions due to the dispersed table structure, which reduces prediction accuracy on both datasets.

6 Conclusion

To fully explore the potential of embedding tables in deep recommender systems, we propose the Rec^2 framework, which enables the model to learn feature individual and interaction information more efficiently from given data. In the Split Stage, we add an identity feature to identify and split feature values that have a stronger relationship to it. In the Aggregate Stage, we use a global heap to determine vital aggregated features more quickly and assign them to new feature fields. Our framework gets superior prediction results on public benchmark datasets, and is expected to provide more accurate recommendations in real-world scenarios.

Acknowledgment

This research was partially supported by the National Natural Science Foundation of China (No.62406303) and Strategic Priority Research Program of the CAS (XDA0490203).

References

[1] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems, Boston, MA, USA, September 15-19, 2016*, Shilad Sen, Werner Geyer, Jill Freyne, and Pablo Castells (Eds.). ACM, 191–198. doi:10.1145/2959100.2959190

[2] Zhaocheng Du, Junhao Chen, Qinglin Jia, Chuhan Wu, Jieming Zhu, Zhenhua Dong, and Ruiming Tang. 2024. LightCS: Selecting Quadratic Feature Crosses in Linear Complexity. In *Companion Proceedings of the ACM on Web Conference 2024*. 38–46.

[3] Zhaocheng Du, Chuhan Wu, Qinglin Jia, Jieming Zhu, and Xu Chen. 2024. A Tutorial on Feature Interpretation in Recommender Systems. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 1281–1282.

[4] Jerome Fan, Suneel Upadhye, and Andrew Worster. 2006. Understanding receiver operating characteristic (ROC) curves. *Canadian Journal of Emergency Medicine* 8, 1 (2006), 19–20.

[5] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. 2019. All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research* 20, 177 (2019), 1–81.

[6] Jingtong Gao, Zhaocheng Du, Xiaopeng Li, Yichao Wang, Xiangyang Li, Huifeng Guo, Ruiming Tang, and Xiangyu Zhao. 2025. SampleLLM: Optimizing Tabular Data Synthesis in Recommendations. In *Companion Proceedings of the ACM on Web Conference 2025*. 211–220.

[7] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, Carles Sierra (Ed.). ijcai.org, 1725–1731. doi:10.24963/ijcai.2017/239

[8] Zai Huang, Zhen Pan, Qi Liu, Bai Long, Haiping Ma, and Enhong Chen. 2017. An Ad CTR Prediction Method Based on Feature Learning of Deep and Shallow Layers. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, CIKM 2017, Singapore, November 06 - 10, 2017*, Ee-Peng Lim, Marianne Winslett, Mark Sanderson, Ada Wai-Chee Fu, Jimeng Sun, J. Shane Culpepper, Eric Lo, Joyce C. Ho, Debora Donato, Rakesh Agrawal, Yu Zheng, Carlos Castillo, Aixin Sun, Vincent S. Tseng, and Chenliang Li (Eds.). ACM, 2119–2122. doi:10.1145/3132847.3133072

[9] Pengyue Jia, Zhaocheng Du, Yichao Wang, Xiangyu Zhao, Xiaopeng Li, Yuhao Wang, Qidong Liu, Huifeng Guo, and Ruiming Tang. 2024. AltFS: Agency-light Feature Selection with Large Language Models in Deep Recommender Systems. *arXiv preprint arXiv:2412.08516* (2024).

[10] Pengyue Jia, Zhaocheng Du, Yichao Wang, Xiangyu Zhao, Xiaopeng Li, Yuhao Wang, Qidong Liu, Huifeng Guo, and Ruiming Tang. 2025. SELF: Surrogate-light Feature Selection with Large Language Models in Deep Recommender Systems. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 1145–1155.

[11] Pengyue Jia, Yejing Wang, Zhaocheng Du, Xiangyu Zhao, Yichao Wang, Bo Chen, Wanyu Wang, Huifeng Guo, and Ruiming Tang. 2024. ERASE: Benchmarking Feature Selection Methods for Deep Recommender Systems. *KDD'2024* (2024).

[12] Zhonghua Jiang, Kui Chen, Kunxi Li, Keting Yin, Yiyun Zhou, Zhaode Wang, Chengfei Lv, and Shengyu Zhang. 2026. Acckv: Towards efficient audio-video llms inference via adaptive-focusing and cross-calibration kv cache optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 5494–5502.

[13] Youngjune Lee, Yeongjong Jeong, Keunchan Park, and SeongKu Kang. 2023. Mvfs: Multi-view feature selection for recommender system. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4048–4052.

[14] Honghao Li, Yiwen Zhang, Yi Zhang, Hanwei Li, Lei Sang, and Jieming Zhu. 2024. FCN: Fusing Exponential and Linear Cross Network for Click-Through Rate Prediction. *arXiv preprint arXiv:2407.13349* (2024).

[15] Honghao Li, Yiwen Zhang, Yi Zhang, Hanwei Li, Lei Sang, and Jieming Zhu. 2026. Fcn: Fusing exponential and linear cross network for click-through rate prediction. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 681–691.

[16] Honghao Li, Yiwen Zhang, Yi Zhang, Lei Sang, and Jieming Zhu. 2025. Revisiting Feature Interactions from the Perspective of Quadratic Neural Networks for Click-through Rate Prediction. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 1365–1375.

[17] Kunxi Li, Zhonghua Jiang, Zhouzhou Shen, ZhaodeWang ZhaodeWang, Chengfei Lv, Shengyu Zhang, Fan Wu, and Fei Wu. 2025. MadaKV: Adaptive Modality-Perception KV Cache Eviction for Efficient Multimodal Long-Context Inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13306–13318.

[18] Kunxi Li, Yufan Xiong, Zhonghua Jiang, Yiyun Zhou, Zhaode Wang, Chengfei Lv, and Shengyu Zhang. 2025. FlowMM: Cross-Modal Information Flow Guided KV Cache Merging for Efficient Multimodal Context Inference. *arXiv preprint arXiv:2511.05534* (2025).

[19] Kunxi Li, Tianyu Zhan, Kairui Fu, Shengyu Zhang, Kun Kuang, Jiwei Li, Zhou Zhao, Fan Wu, and Fei Wu. 2025. Mergenet: Knowledge migration across heterogeneous models, tasks, and modalities. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 4824–4832.

[20] Jianghao Lin, Yanru Qu, Wei Guo, Xinyi Dai, Ruiming Tang, Yong Yu, and Weinan Zhang. 2023. Map: A model-agnostic pretraining framework for click-through rate prediction. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1384–1395.

[21] Weilin Lin, Xiangyu Zhao, Yejing Wang, Tong Xu, and Xian Wu. 2022. AdaFS: Adaptive feature selection in deep recommender system. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3309–3317.

[22] Qi Liu, Zhilong Zhou, Gangwei Jiang, Tiezheng Ge, and Defu Lian. 2023. Deep task-specific bottom representation network for multi-task recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1637–1646.

[23] Zirui Liu, Jiatong Li, Yan Zhuang, Qi Liu, Shuanghong Shen, Jie Ouyang, Mingyue Cheng, and Shijin Wang. 2025. am-ELO: A Stable Framework for Arena-based LLM Evaluation. In *Forty-second International Conference on Machine Learning*.

[24] Ze Liu, Xianquan Wang, Shuochen Liu, Jie Ma, Huibo Xu, Yupeng Han, Zhe Yang, Kai Zhang, Longfei Li, and Jun Zhou. 2025. CrossDenoise: Denoising Implicit Feedback via a Lightweight Entity-Aware Synergistic Framework. *arXiv preprint arXiv:2508.10851* (2025).

[25] Zirui Liu, Xianquan Wang, Yan Zhuang, Jiatong Li, Qi Liu, Shuanghong Shen, Mingyue Cheng, and Shijin Wang. 2026. Fewer Battles, More Gain: An Information-Efficient Framework for Arena-based LLM Evaluation. In *The Fourteenth International Conference on Learning Representations*.

[26] Zirui Liu, Yan Zhuang, Qi Liu, Jiatong Li, Yuren Zhang, Zhenya Huang, Jinze Wu, and Shijin Wang. 2024. Computerized adaptive testing via collaborative ranking. *Advances in Neural Information Processing Systems* 37 (2024), 95488–95514.

[27] Xupeng Miao, Yining Shi, Hailin Zhang, Xin Zhang, Xiaonan Nie, Zhi Yang, and Bin Cui. 2022. HET-GMP: a graph-based system approach to scaling large embedding model training. In *Proceedings of the 2022 International Conference on Management of Data*. 470–480.

[28] Ping Ni, Xianquan Wang, Bing Lv, and Likang Wu. 2024. GTR: An explainable Graph Topic-aware Recommender for scholarly document. *Electronic Commerce Research and Applications* 67 (2024), 101439. doi:10.1016/j.elerap.2024.101439

[29] Yanru Qu, Han Cai, Kan Ren, Weinan Zhang, Yong Yu, Ying Wen, and Jun Wang. 2016. Product-based neural networks for user response prediction. In *2016 IEEE 16th international conference on data mining (ICDM)*. IEEE, 1149–1154.

[30] Fangye Wang, Hansu Gu, Dongsheng Li, Tun Lu, Peng Zhang, and Ning Gu. 2023. Towards deeper, lighter and interpretable cross network for ctr prediction. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 2523–2533.

[31] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*. 1–7.

[32] Xianquan Wang, Zhaocheng Du, Jieming Zhu, Qinglin Jia, Zhenhua Dong, and Kai Zhang. 2026. FairFS: Addressing Deep Feature Selection Biases for Recommender System. In *Proceedings of The Web Conference 2026*. To appear. doi:10.1145/3774904.3792167

[33] Xianquan Wang, Zhaocheng Du, Jieming Zhu, Chuhan Wu, Qinglin Jia, and Zhenhua Dong. 2025. TayFCS: Towards Light Feature Combination Selection for Deep Recommender Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 5007–5017.

[34] Xianquan Wang, Likang Wu, Zhi Li, Haitao Yuan, Shuanghong Shen, Huibo Xu, Yu Su, and Chenyi Lei. 2025. Mitigating redundancy in deep recommender systems: A field importance distribution perspective. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 1515–1526.

[35] Yejing Wang, Zhaocheng Du, Xiangyu Zhao, Bo Chen, Huifeng Guo, Ruiming Tang, and Zhenhua Dong. 2023. Single-shot feature selection for multi-task recommendations. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 341–351.

[36] Yejing Wang, Xiangyu Zhao, Tong Xu, and Xian Wu. 2022. Autofield: Automating feature selection in deep recommender systems. In *Proceedings of the ACM Web Conference 2022*. 1977–1986.

[37] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2023. A survey on large language models for recommendation. *arXiv preprint arXiv:2305.19860* (2023).

[38] Xiang Xu, Hao Wang, Wei Guo, Luankang Zhang, Wanshan Yang, Runlong Yu, Yong Liu, Defu Lian, and Enhong Chen. 2025. Multi-granularity interest retrieval and refinement network for long-term user behavior modeling in ctr prediction. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 2745–2755.

[39] Hengyu Zhang, Junwei Pan, Dapeng Liu, Jie Jiang, and Xiu Li. 2024. Deep Pattern Network for Click-Through Rate Prediction. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1189–1199.

[40] Haotian Zhang, Shuanghong Shen, Bihan Xu, Zhenya Huang, Jinze Wu, Jing Sha, and Shijin Wang. 2024. Item-difficulty-aware learning path recommendation: From a real walking perspective. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4167–4178.

[41] Kai Zhang, Hao Qian, Qing Cui, Qi Liu, Longfei Li, Jun Zhou, Jianhui Ma, and Enhong Chen. 2021. Multi-interactive attention network for fine-grained feature learning in ctr prediction. In *Proceedings of the 14th ACM international conference on web search and data mining*. 984–992.

[42] Yiyun Zhou, Zhonghua Jiang, Wenkang Han, Kunxi Li, Mingjing Xu, Chang Yao, and Jingyuan Chen. 2026. A _3 B _2: Adaptive Asymmetric Adapter for Alleviating Branch Bias in Vision-Language Image Classification with Few-Shot Learning. *arXiv preprint arXiv:2605.13161* (2026).

[43] Jieming Zhu, Qinglin Jia, Guohao Cai, Quanyu Dai, Jingjie Li, Zhenhua Dong, Ruiming Tang, and Rui Zhang. 2023. Final: Factorized interaction layer for ctr prediction. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2006–2010.

[44] Jieming Zhu, Jinyang Liu, Shuai Yang, Qi Zhang, and Xiuqiang He. 2021. Open benchmarking for click-through rate prediction. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 2759–2769.